NIT2112 AI Copilot Prompts Guide

Effective Prompts for OOP Practical Assessments

Introduction

This guide provides a comprehensive collection of AI Copilot prompts for completing NIT2112 Object-Oriented Programming practical assessments. It includes example prompts from the lab materials and additional generated prompts organised by OOP concept and development phase.

Important: Using AI Copilots effectively means asking targeted questions, critically evaluating responses, and adapting suggestions to your specific design. Never blindly copy AI-generated code without understanding it.

Section 1: Example Prompts from Lab Materials

1.1 Documentation Prompts (from Lab 10)

Docstring Generation:

```
"Generate a detailed Python docstring for this method using Google style
format. Include descriptions for parameters (Args), return values
(Returns), and any custom exceptions it might raise (Raises)."
```

Code Comments:

```
"Add explanatory code comments to this Python code block to clarify the
logic, especially the purpose of different checks or error handling."
```

System Summary:

```
"Write a short summary describing the purpose and relationships between the
main classes (Person, Student, Teacher, Unit, EnrollmentSystem,
SchoolRegistry) in my Python school simulator project."
```

Interaction Explanation:

```
"Explain, in plain English, the steps involved when a student enrolls in a
unit using the EnrollmentSystem, mentioning the roles of the Registry and
the Unit class."
```

Diagram Generation:

```
"Generate PlantUML (or Mermaid) syntax for a simple class diagram. It
should show that Student and Teacher inherit from an abstract Person class.
It should also show an association relationship between Student and Unit."
```

README Draft:

```
"Generate a draft README.md file for my Python project. Include sections
for: Project Title, Description, Features, Getting Started, Main Classes,
and How to Run Tests."
```

1.2 Practical Assessment Prompts (Good Examples)

Design Trade-offs:

```
"What are the trade-offs between using composition vs inheritance for my
Zone classes?"
```

Pattern Implementation:

```
"Help me implement an Observer pattern for low-stock alerts where products
notify subscribed managers."
```

Refactoring:

```
"How can I refactor this code to better follow the Single Responsibility
Principle?"
```

Debugging Polymorphism:

```
"Why might my polymorphic is_shippable() method not be called on
PerishableProduct?"
```

Ticket Validation:

```
"Why is my EarlyBirdTicket validation not correctly checking the 30-day
rule?"
```

1.3 Poor AI Usage Examples (What to Avoid)

■ **Wholesale Delegation:**

```
""Write the entire warehouse system for me""
```

Why it's poor: Too broad; shows no understanding of requirements

■ **No Critical Thinking:**

```
"Accepting generated code without understanding or testing it"
```

Why it's poor: Must always evaluate and adapt AI suggestions

■ **No Documentation:**

```
"Using AI but not recording how it influenced your solution"
```

Why it's poor: AI interactions must be documented for assessment

■ **Ignoring Context:**

```
"Pasting AI code that doesn't integrate with your design"
```

Why it's poor: AI suggestions must fit your architecture

Section 2: Design Phase Prompts

2.1 Understanding Requirements

```
"Help me break down these business requirements into potential classes and their responsibilities."

"What entities should I model for a [domain] system? List the key nouns that could become classes."

"Looking at these requirements, which relationships should be composition vs aggregation?"

"What are the main actors and objects in this system? How do they interact?"

"Help me identify the core abstractions needed for this problem domain."
```

2.2 Class Hierarchy Design

```
"Should I use inheritance or composition for the relationship between [ClassA] and [ClassB]? What are the trade-offs?"

"I have three types of [Entity]: Type1, Type2, and Type3. How should I structure the inheritance hierarchy?"

"What common attributes and methods should go in my base [Entity] class vs the subclasses?"

"Is this a good candidate for an abstract base class, or should it be a concrete class?"

"How deep should my inheritance hierarchy be? I currently have 4 levels."

"What's the difference between using ABC vs Protocol for defining interfaces in Python?"
```

2.3 Design Pattern Selection

```
"Which design pattern would be best for creating different types of [Entity] based on configuration?"

"I need to notify multiple objects when [Event] occurs. Which pattern should I use?"

"I have legacy data in format X and need to convert it to format Y. Is the Adapter pattern appropriate?"

"How do I ensure only one instance of my [Registry/Manager] class exists? Show me the Singleton pattern in Python."

"Should I use Factory Method or Abstract Factory for creating my product hierarchy?"

"I need to add functionality to objects at runtime without modifying them. Which pattern fits?"
```

Section 3: Implementation Phase Prompts

3.1 Encapsulation

"How do I implement private attributes with property getters and setters in Python?"

"Show me how to add validation to a property setter that ensures [condition]."

"What's the Python convention for protected vs private attributes? When should I use each?"

"How do I make an attribute read-only after initialization?"

"Should this be a property or a method? It requires some calculation."

3.2 Abstract Base Classes & Interfaces

"Show me how to define an abstract base class with abstract methods using Python's abc module."

"How do I create an interface-like class that defines a contract for [Behavior]?"

"Can a class implement multiple interfaces in Python? Show me an example."

"When should I use @abstractmethod vs @abstractproperty?"

"How do I enforce that subclasses implement certain methods?"

3.3 Polymorphism

"Help me implement polymorphic behavior where each [Entity] type calculates [value] differently."

"Show me how to override a method in a subclass while still calling the parent's implementation."

"How do I use duck typing effectively in Python for polymorphism?"

"What's the difference between method overriding and method overloading in Python?"

"How do I implement __str__ and __repr__ methods that are polymorphic across my hierarchy?"

3.4 Factory Pattern Implementation

"Show me how to implement a Factory class that creates different [Entity] types based on a type string."

"How do I add validation in my factory to ensure required parameters are provided?"

"Should my factory use class methods or static methods?"

"How do I extend my factory to support new product types without modifying existing code?"

"What's the best way to handle invalid type parameters in a factory?"

3.5 Observer Pattern Implementation

```
"Help me implement the Observer pattern where [Subject] notifies [Observers] when [Event] occurs."

"How do I manage multiple types of notifications (e.g., email, SMS, dashboard) using Observer?"

"Show me how to implement add_observer, remove_observer, and notify_observers methods."

"How do I pass different types of data to observers depending on the event type?"

"Should observers pull data from the subject or should the subject push data to observers?"
```

3.6 Adapter Pattern Implementation

```
"How do I create an Adapter class to convert legacy data format to my new system's format?"

"Show me how to adapt a class with interface X to work with interface Y."

"What's the difference between object adapter and class adapter patterns?"

"How do I handle data type conversions (e.g., cents to dollars, grams to kg) in an adapter?"

"My legacy system uses different field names. How do I map them in the adapter?"
```

3.7 Singleton Pattern Implementation

```
"Show me how to implement the Singleton pattern in Python using a metaclass."

"What are the different ways to implement Singleton in Python and their trade-offs?"

"How do I make my [Registry] class a Singleton to ensure only one instance exists?"

"Is there a way to reset a Singleton for testing purposes?"

"How do I handle thread safety with the Singleton pattern in Python?"
```

Section 4: Exception Handling Prompts

4.1 Custom Exception Design

"Help me design a custom exception hierarchy for my [Domain] system."

"What exceptions should I define for handling [specific business rules]?"

"How do I add context information (like entity IDs) to my custom exceptions?"

"Should I have separate exceptions for validation errors vs business rule violations?"

"Show me how to create a base exception class that all my domain exceptions inherit from."

4.2 Exception Implementation

"How do I implement a custom exception with additional attributes like error_code and details?"

"Show me how to chain exceptions in Python to preserve the original error context."

"When should I raise an exception vs return an error code or None?"

"How do I write informative exception messages that help with debugging?"

"What's the best practice for documenting exceptions in docstrings?"

Section 5: Debugging Prompts

5.1 Common OOP Issues

"Why is my abstract method not being called on the subclass?"

"My isinstance() check is returning False when I expect True. What could cause this?"

"Why is my property setter not being called when I assign a value?"

"My Observer is not receiving notifications. How do I debug this?"

"The Singleton is creating multiple instances. What's wrong with my implementation?"

5.2 Debugging Specific Patterns

"My factory is returning None instead of the created object. Why?"

"The adapter is not converting the data correctly. Here's my code: [paste code]"

"Why is super().__init__() not calling the parent constructor as expected?"

"My method override is not being recognized. Is there something wrong with the signature?"

```
"How do I trace method resolution order (MRO) issues in my class
hierarchy?"
```

5.3 Logic Errors

```
"My validation is passing when it should fail. Here's the condition: [paste
code]"

"The calculation in my [method] is returning wrong values. Can you spot the
error?"

"Why is my loop not iterating over all items in the collection?"

"My date comparison is not working correctly. What's wrong?"

"The discount is being applied incorrectly. Here's my pricing logic: [paste
code]"
```

Section 6: Refactoring Prompts

6.1 Code Quality Improvements

```
"How can I refactor this long method into smaller, more focused methods?"

"This class has too many responsibilities. How should I split it?"

"How do I refactor duplicate code in these two classes into a shared base class?"

"What's a cleaner way to handle these multiple if-elif conditions?"

"How can I make this code more Pythonic?"
```

6.2 SOLID Principles

```
"How can I refactor this code to follow the Single Responsibility Principle?"

"My class is violating the Open/Closed Principle. How do I fix it?"

"Is this a Liskov Substitution Principle violation? How should I restructure?"

"How do I apply the Interface Segregation Principle to this large interface?"

"Help me apply Dependency Inversion to reduce coupling between these classes."
```

Section 7: Domain-Specific Prompts

7.1 Warehouse/Inventory Systems

```
"How should I model different product types (standard, perishable, hazardous) in a warehouse system?"

"What's the best way to implement stock level tracking with low-stock alerts?"

"How do I handle product expiry dates and prevent shipping of expired items?"

"Show me how to implement zone-based storage with capacity limits."

"How should I design order fulfillment that deducts stock from multiple zones?"
```

7.2 Event Management Systems

```
"How do I model different event types (conference, workshop, webinar) with their specific rules?"

"What's the best approach for implementing tiered ticket pricing (standard, VIP, early bird)?"

"How should I handle event capacity and waitlist management?"
```

"Show me how to implement registration with duplicate email prevention."

"How do I calculate ticket prices with various discounts and multipliers?"

7.3 Healthcare/Clinic Systems

"How should I model patient types (outpatient, inpatient, emergency) with different billing rules?"

"What's the best way to implement appointment scheduling with slot validation?"

"How do I handle medical record access control and privacy?"

"Show me how to implement insurance calculation and coverage rules."

"How should I design prescription tracking with medication interaction checks?"

7.4 Booking/Reservation Systems

"How do I model room types with seasonal pricing variations?"

"What's the best approach for implementing booking status transitions?"

"How should I handle cancellation policies with graduated refunds?"

"Show me how to implement loyalty points earning and redemption."

"How do I prevent double-booking and handle conflicts?"

7.5 Financial/Banking Systems

"How should I model different account types with their interest rates and limits?"

"What's the best way to implement transaction validation and daily limits?"

"How do I handle overdraft protection and linked accounts?"

"Show me how to implement interest calculation (daily accrual, monthly application)."

"How should I design ATM operations with PIN validation and card blocking?"

Section 8: AI Interaction Logging

When documenting AI interactions for your reflection document, include:

1. **The prompt you used** — Record the exact question or request
2. **Summary of the AI's response** — Key points, code suggestions, or explanations
3. **How you used/modified/rejected the suggestion** — Critical evaluation
4. **Why you made that decision** — Your reasoning and understanding

Example AI Interaction Log Entry:

> **Prompt:** "How should I implement the Observer pattern for low-stock alerts?" **AI Response Summary:** The AI suggested creating an Observable interface with add_observer(), remove_observer(), and notify_observers() methods. Products inherit this interface and call notify_observers() when quantity drops below threshold. **How I Used It:** I adopted the basic structure but modified it to: - Pass the product SKU and current quantity in the notification - Use a default threshold of 10 instead of requiring it as a parameter - Added AlertType enum to categorize different notification types **Why:** The AI's suggestion was good for the basic pattern, but I needed more context in notifications for the WarehouseManager to take action. Adding the enum makes the system more extensible for future alert types.

Section 9: Prompt Engineering Tips

Be Specific:

> *Instead of 'help me with classes', say 'help me design a class hierarchy for product types where each has different shipping rules'*

Provide Context:

> *Include relevant code snippets, class names, and the problem domain*

Ask for Trade-offs:

> *Request pros/cons when choosing between approaches*

Request Examples:

> *Ask for Python code examples demonstrating the concept*

Iterate:

> *Build on previous responses with follow-up questions*

Challenge Assumptions:

> *Ask 'what if' questions to explore edge cases*

Request Alternatives:

> *Ask for multiple approaches to compare*

Verify Understanding:

> *Ask the AI to explain its suggestion in simpler terms*

This guide was created as a reference for NIT2112 Object-Oriented Programming. Use these prompts as starting points and adapt them to your specific assessment requirements.